

se ho un automa  $M = \{Q, \Sigma, \delta, q_0, F\}$  si dice che  $M$  accetta una stringa  $w \in \Sigma^*$  se esiste una sequenza di stati:

$$R_0, R_1, R_2, \dots, R_m \in Q$$

Tali che:

1) il primo stato deve coincidere con lo stato iniziale per l'automa  $R_0 = q_0$

2)  $\forall i = 0, \dots, m-1 \quad R_{i+1} \in \delta(R_i, y_{i+1}) \rightarrow y_{i+1} = \{ \epsilon \} \text{ oppure } y_{i+1} = w_i$

↳ successivo simbolo da leggere da parte dell'automa!

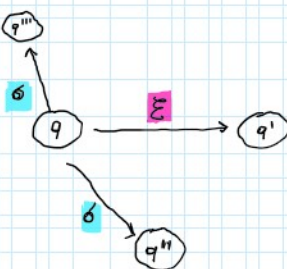
3) l'ultimo stato deve fare parte degli stati di accettazione dell'automa

$$R_m \in F$$

In questo tipo di automa, l'automa non è obbligato a consumare un simbolo dell'input per ogni cambiamento di stato/TRANSIZIONE.

alcuni cambiamenti di stato possono essere fatti anche senza consumare alcun input, cioè quando  $y_{i+1}$  è la stringa vuota.

Esempio



•  $q$  va verso  $q'$  senza consumare un input

• L'automa potrebbe TRANSITARE da  $q$  a  $q''$  consumando un input

questo rende l'automa **Non deterministico**,

perché l'automa può scegliere se TRANSITARE autonomamente a  $q'$  oppure scegliere di consumare il simbolo  $0$  e TRANSITARE a  $q''$ .

L'automa non deterministico sta modellando una forma di COMPUTAZIONE **parallela**.

• dallo stato  $q$ , si potrebbe ARRIVARE allo stato  $q''$  tramite lo stesso simbolo  $0$  che portava a  $q'$ .

L'automa, dallo stato  $q$ , leggendo il simbolo  $0$ , può decidere se passare a  $q''$  o a  $q'$ , ed è come se portasse avanti una computazione in parallelo facendolo ENTRAMBE le cose.

In un **DFA** non c'è possibilità di scelta: ogni stato ED ogni simbolo letto determinano lo stato successivo, non è possibile "non leggere un simbolo" e la computazione va avanti fino alla fine.

In un **NFA** invece c'è molta più flessibilità: l'automa accetta se esiste almeno una sequenza di stati che rispetta le regole e che porta allo stato di accettazione.

L'automa cerca la sequenza ideale in parallelo, se esiste viene trovata, e se viene trovata, accetta la stringa.

